

Assignment - 1

1. Discuss the various methods provided by the Array class in Java & illustrate the usage of these methods with a suitable example program.

A:- 1. sort():- sorts the array elements in ascending order. overloaded versions support sorting primitive & object Array.

Ex:- `Arrays.sort(arr);`

2. Binary Search():- performs binary search on a sorted array. returns the index of the key if found; otherwise returns insertion point - 1.

Ex:- `Arrays.binarySearch(arr, key);`

3. equals():- compares two array element by element. returns true if all elements match.

Ex:- `Arrays.equals(arr1, arr2);`

4. fill():- fills the whole array with a specific value. fills all element with 0.

Ex:- `Arrays.fill(arr1, 0);`

5. copyOf():- copy the original array to a new array of given length.

Ex:- `Arrays.copyOf(arr, newlength);`

6. toString():- returns a string representation of the array.

Ex:- `Array.toString(arr);`

7. copyOfRange():- copies a specific range from the original array.

Ex:- `Arrays.copyOfRange(arr, from, to);`

8. asList():- converts array to a fixed size list.

Ex:- `Arrays.asList(arr);`

program :-

```
import java.util. Arrays;
```

```
public class ArraysDemo {
```

```
    public static void main(String args[]) {
```

```
        int[] arr = { 5, 2, 8, 1, 3 };
```

```
        Arrays.sort(arr);
```

```
        System.out.println("Sorted array = " + Arrays.toString(arr));
```



```

int index = Arrays.binarySearch(arr, 3);
System.out.println("Index of 3: " + index);
Arrays.fill(arr, 0);
System.out.println("Array after fill: " + Arrays.toString(arr));

```

```

int[] newArr = Arrays.copyOf(arr, 3);
System.out.println("Copied Array" + Arrays.toString(arr));
boolean isEqual = Arrays.equals(arr, newArr);
System.out.println("Are boolean equal? " + isEqual);
}

```

O/p:- Sorted array: [1, 2, 3, 4, 5, 8]
 Index of 3: 2
 Array after fill: [0, 0, 0, 0, 0, 0]
 Copied Array: [0, 0, 0, 0]
 Are boolean equal? false.

2. Define a comparator. mention the methods provided by the comparator interface. illustrate its use with a program that demonstrates sorting elements in a TreeSet in reverse order.
- Ans:- Comparator interface:- A comparator is an interface in java.util used to define custom ordering for objects. It allows us to compare two objects & decide their order.
- ```

1. int compare(T a, T b)

```
- \* if compares two objects and returns
  - \* -ve value if a should come before or 0 if a & b are equal
  - \* +ve value if a should come after b.
- Ex:- Comparator < Integer > comp = new Comparator<Integer>() {  
 public int compare(Integer a, Integer b) {

return a-b; } };

2. boolean equals (Object obj)
- \* checks if two comparator objects are equal
- Ex:- Comparator < String > c1 = (s1, s2) -> s1.length() - s2.length();  
 Comparator < String > c2 = (s1, s2) -> s1.length() - s2.length();  
 System.out.println(c1.equals(c2));
3. Static < T > Comparator < T > reversed()
- \* used to reverse the natural order (ascending -> descending)
- Ex:- Comparator < Integer > comp = Comparator.reverseOrder();  
 default Comparator < T > then comparing (....)
- \* used to chain multiple comparisons

Ex:- Comparator < Student > comp = Comparator  
 -> comparing (Student::getMarks);  
 -> then comparing (Student::getName);  
 TreeSet with Comparator program:-  
 import java.util.\*;  
 class ReverseOrderComparator implements Comparator<String> {  
 public int compare(String a, String b) {  
 return a.compareTo(b); }  
 public class TreeSetReverseSort {  
 public static void main (String args[]) {  
 TreeSet<String> ts = new TreeSet<> (new ReverseOrder  
 Comparator());  
 ts.add("C");  
 ts.add("P");  
 ts.add("B");  
 System.out.println("TreeSet in reverse order: " + ts);  
 }  
 }  
 o/p:- [C, B, A]



3. Describe all the String Comparison methods available in java with example each?

Ans: 1. equals(): To compare two strings for equality. we equals() has the general form.

boolean equals(Object str)

1. equalsIgnoreCase(): This comparison is case-sensitive to perform a comparison that ignores case difference call equalsIgnoreCase()
2. regionMatches(): Compares a specific region inside a string with another specific region in another string
3. startsWith(): determines whether a given string begins with a specified string.
4. endsWith(): determines whether string ends with a specified string
5. equalsTo (a =): Compares string by string.
6. compareTo(): Same as equals but it will return and put in the form of 0 & 1.

Ex:- import java.util.\*;

import java.io.\*;

class StringDemo {

public static void main (String args[]) {

String s1 = "Sham is a boy";

String s2 = "Sham";

System.out.println(s1.regionMatches(0, s2, 0, s2.length()));

System.out.println(s1.startsWith("S"));

System.out.println(s1.equals(s2));

s1 = Sham, s2 = sham

System.out.println(s1 == s2);

System.out.println(s1.compareTo(s2));

}

o/p:- false

false

false

true, 1

4. Demonstrate the usage of the String modification methods given with example each.

Ans: 1. substring(): To extract substring using substring() method. It has 2 forms.

1. String substring(int startindex)
2. String substring(int startindex, int endindex)
3. concat(): we can concatenate two strings using concat(). String concat (String str)
4. trim(): returns a copy of invoking string from which any leading & trailing spaces are removed.
5. replace(): replace() method has two forms. the first replace all occurrences of one character.
6. String replace (char original, char replacement)
7. String replace (char sequence original, char sequence replace)

Ex:- import java.util.\*;

import java.io.\*;

class StringDemo {

public static void main (String args[]) {

s1 = "Ravi";

s2 = "Ravi";

System.out.println(s1.concat(s2));

System.out.println(s2.replace('1', '2'));

System.out.println(s1.substring(0, 2));

s1 = "Ravi", s2 = "Ravi"

System.out.println(s1.trim());

}

o/p:- RaviRavi

Ravi

Ravi

Ravi



5. Explain the usage of `indexOf()` and `lastIndexOf()` method with one example each.

Ans: 1. `indexOf()` :- `int indexOf(String str);`

Ex:- `StringBuffer sb = new StringBuffer("Hello world");`

`System.out.println(sb.indexOf("world"));`

o/p :- 6

`int indexOf(String str, int startindex)`

`System.out.println(sb.indexOf("ello", 4));`

Searches the starting string buffer for the first occurrence of string beginning at the start index returns the index of the match or -1 if no match found.

2. `lastIndexOf()` :- `int lastIndexOf(String str);`

`StringBuffer sb = new StringBuffer("Hello");`

`System.out.println(sb.lastIndexOf("Hello"));`

o/p :- 9

Searches the starting string buffer for the last occurrence of the str it returns the index of the match or -1 not found.

6. Illustrate the use of `StringBuffer` methods: `append()`, `insert()`, `reverse()`, `capacity()` & `delete()` with proper example.

Ans: 1. `append()` :- the append method concatenates the string representation of any other type of data to the end of the `StringBuffer`.

- of string object

2. `insert()` :- the insert method inserts one string into another.

There are four forms :-

- `StringBuffer insert(int index, String str)`
- `StringBuffer insert(int index, char ch)`
- `StringBuffer insert(int index, Object obj)`

8. `reverse()` :- you can reverse the characters within a string buffer.

- object using reverse.

4. `capacity()` :- total allocated capacity can be found through `capacity()` method.

`int capacity()`

5. `delete()` :- you can delete characters within the `StringBuffer` by using the methods `delete()` & `deleteCharAt()`.

- `StringBuffer delete(int start index, int end index)`

Ex:- `import java.util.*;`

`import java.io.*;`

`class StringDemo {`

`public static void main(String args[]) {`

`StringBuffer sb = new StringBuffer("Manish Pawar");`

`System.out.println(sb.capacity());`

`String c = sb.append("Manish").append("Pawar").toString();`

`-g();`

`System.out.println(c);`

`sb.insert(8, "Mani");`

`sb.insert(7, "n");`

`sb.insert(0, c);`

`sb.delete(0, 1);`

`sb = new StringBuffer("Mani");`

`sb.reverse();`

`}`

o/p :- 28

Manish Pawar ManishPawar  
Manish PManishPawarManishPawar  
Manish nPManishPawarManishPawar  
Manish PawarManishPawarManish  
nPManishPawarManishPawar



anish Pawar  
Mansi Pawar  
Mansi Pawar

Q. What are legacy classes? Explain any two legacy classes of Java Collection Framework with suitable programs.

Ans: A legacy class is an old Java class that was created using outdated design or technology but is still used in modern applications.

1. Vector: A resizable array or general purpose dynamic array.

Package: java.util

Constructor: Vector(), Vector(int size, Object), Vector(Collection <? extends E> c);

Methods: extendAbstractList, implementList, randomAccess().

1. addElement(): Adds element at the end.

2. elementAt(): Returns element at given index.

3. insertElementAt(): Inserts element at specific position.

4. size(): Returns number of elements.

Example: import java.util.\*;

class legacyDemo {

public static void main(String args[]) {

Vector v1 = new Vector();

v1.addElement("Sugil");

v1.addElement(8);

v1.addElement(9.5);

System.out.println(v1);

System.out.println(v1.indexOf(8));

System.out.println(v1.indexOf("Siddu", 5));

System.out.println(v1);

o/p: [Muzam1, Sugil, 8.5, null, 9.5]

2. Stack: A LIFO (Last In First Out) data structure.

Package: java.util

Constructor: Stack();

Methods: 1. push(): Adds element on top.

2. peek(): Returns top element without removing.

3. empty(): Checks if stack is empty.

Example: import java.util.\*;

class legacyDemo {

public static void main(String args[]) {

Stack s1 = new Stack();

s1.push("Muzam1");

s1.push("Sannan");

s1.push(70);

s1.push(30);

s1.push(-1);

System.out.println(s1);

System.out.println(s1.pop());

System.out.println(s1.peek());

System.out.println("Stack size is " + s1.size());

o/p: [Muzam1, Sannan, 70, 30, -1]

Stack size is 4

3. Explain the Collection Algorithms?

Ans:- Collections Framework defines several algorithms that can be applied to collections & maps.

\* `addAll()` :- Adds elements to a Collection; returns true if added.

\* `addAllQueue()` :- Converts deque into stack (LIFO).

\* `BinarySearch(list, value, comparator)` :- Searches element using comparator; returns index or negative if not found.

\* `BinarySearch(list, value)` :- Searches in sorted list; returns index or negative if not found.

\* `checkedCollection()` :- Type-safe collection.

\* `checkedList()` :- Type-safe list.

\* `checkedMap()` :- Type-safe map (key & value checked).

\* `checkedSet()` :- Type-safe set.

\* `frequency()` :- Counts occurrences of an element.

\* `indexOfSublist()` :- Finds first occurrence of sublist.

\* `lastIndexOfSublist()` :- Finds last occurrence of sublist.

Ex:- import java.util.\*;

class AlgorithmDemo {

public static void main(String args[]) {

LinkedList <Integer> ll = new LinkedList <Integer> ();

ll.add(81);

ll.add(20);

ll.add(20);

ll.add(81);

Comparator <Integer> r = Collections.reverseOrder();

Collections.sort(ll, r);

System.out.println("list sorted in reverse:");

for (int i: ll)

System.out.println(i + " ");

System.out.println();

Collections.shuffle(ll);

System.out.println("list shuffled:");

for (int i: ll)

System.out.println(i + " ");

System.out.println();

System.out.println("Maximum: " + Collections.max(ll));

(ll);

} }

o/p:- list sorted in reverse: 20 8 -8 20

list shuffled: 20 -20 8 -8

Maximum: 20

Minimum: -20